

# TWO POINTERS – PROBLEMS-PHASE-I

## TWO POINTERS – PROBLEM 1: TWO SUM II (INPUT ARRAY IS SORTED)

### Problem Description

Given a **1-indexed sorted array** of integers `numbers` and an integer `target`, find two numbers such that they add up to the target.

Return the indices of the two numbers (1-based indexing).

You may assume that exactly one solution exists.

### Sample Input

```
numbers = [2,7,11,15]
target = 9
```

### Sample Output

```
[1,2]
```

### Constraints

```
2 <= numbers.length <= 3 * 10^4
-1000 <= numbers[i] <= 1000
numbers is sorted in non-decreasing order
-1000 <= target <= 1000
```

### Brute Force Approach

#### Idea

Check every possible pair and return the pair whose sum equals the target.

#### Algorithm

1. Traverse each element.
2. For every element, check all remaining elements.
3. If sum equals target, return indices.

#### Time Complexity

$O(n^2)$

#### Space Complexity

$O(1)$

### Optimized Two Pointer Approach

#### Idea

Since the array is already sorted:

- Place one pointer at the beginning.
- Place another pointer at the end.
- Compare their sum with target.

#### Algorithm

1. Initialize:
  - `left = 0`
  - `right = n - 1`
2. Compute sum.
3. If `sum == target` → return answer.
4. If `sum < target` → move left pointer.
5. If `sum > target` → move right pointer.

#### Time Complexity

$O(n)$

#### Space Complexity

$O(1)$

### Company Tags

Amazon, Google, Microsoft, Adobe

### Practice Link

<https://leetcode.com/problems/two-sum-ii-input-array-is-sorted/>

### Video Solution

<https://www.youtube.com/watch?v=cQ1Oz4ckceM>

### Java Solution

```
class Solution {
    public int[] twoSum(int[] numbers, int target)
    {

        int left = 0;
        int right = numbers.length - 1;

        while(left < right) {

            int sum = numbers[left] +
            numbers[right];

            if(sum == target) {
                return new int[]{left + 1, right +
            1};
            }

            if(sum < target) {
                left++;
            } else {
                right--;
            }
        }

        return new int[]{-1, -1};
    }
}
```

### Dry Run

```
numbers = [2,7,11,15]
target = 9
```

```
2 + 15 = 17 > 9
Move right
```

```
2 + 11 = 13 > 9
Move right
```

```
2 + 7 = 9
Answer = [1,2]
```

# TWO POINTERS – PROBLEM 2:

## VALID PALINDROME

### Problem Description

Given a string `s`, determine whether it is a palindrome after converting all uppercase letters to lowercase and removing all non-alphanumeric characters.

### Sample Input

`s = "A man, a plan, a canal: Panama"`

### Sample Output

`true`

### Constraints

`1 <= s.length <= 2 * 10^5`

### Brute Force Approach

#### Idea

Create a cleaned string, reverse it, and compare.

#### Time Complexity

$O(n)$

#### Space Complexity

$O(n)$

### Optimized Two Pointer Approach

#### Idea

Use two pointers from both ends.

Skip non-alphanumeric characters and compare characters directly.

#### Algorithm

1. `left = 0`
2. `right = n - 1`
3. Skip invalid characters.
4. Compare characters.
5. Move pointers inward.

#### Time Complexity

$O(n)$

#### Space Complexity

$O(1)$

### Company Tags

Facebook, Amazon, Microsoft

### Practice Link

<https://leetcode.com/problems/valid-palindrome/>

### Video Solution

<https://www.youtube.com/watch?v=jJXJ16kPFWg>

### Java Solution

```
class Solution {
    public boolean isPalindrome(String s) {

        int left = 0;
        int right = s.length() - 1;

        while(left < right) {

            while(left < right &&
!Character.isLetterOrDigit(s.charAt(left))) {
                left++;
            }

            while(left < right &&
```

```
!Character.isLetterOrDigit(s.charAt(right))) {
                right--;
            }

            if(Character.toLowerCase(s.charAt(left))
!=
Character.toLowerCase(s.charAt(right))) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }
}
```

### Dry Run

Input:

"A man, a plan, a canal: Panama"

Compare:

`a = a`

`m = m`

`a = a`

`n = n`

All matched

Output = true

# TWO POINTERS –

## PROBLEM 3: CONTAINER WITH MOST WATER

### Problem Description

Given an integer array `height`, find two lines that together with the x-axis form a container that stores the maximum amount of water.  
Return the maximum area.

### Sample Input

```
height = [1,8,6,2,5,4,8,3,7]
```

### Sample Output

```
49
```

### Constraints

```
2 <= n <= 10^5  
0 <= height[i] <= 10^4
```

### Brute Force Approach

#### Idea

Check every pair of lines and calculate area.

#### Time Complexity

```
O(n^2)
```

#### Space Complexity

```
O(1)
```

### Optimized Two Pointer Approach

#### Idea

Use left and right pointers.

Move the pointer having smaller height.

#### Why?

The area is limited by the shorter wall.

Increasing the shorter wall may increase area.

#### Time Complexity

```
O(n)
```

#### Space Complexity

```
O(1)
```

### Company Tags

Google, Amazon, Adobe, Bloomberg

### Practice Link

<https://leetcode.com/problems/container-with-most-water/>

### Video Solution

<https://www.youtube.com/watch?v=ZHQhCs7Kdlc>

### Java Solution

```
class Solution {  
    public int maxArea(int[] height) {  
  
        int left = 0;  
        int right = height.length - 1;  
  
        int maxArea = 0;  
  
        while(left < right) {  
  
            int width = right - left;  
            int h = Math.min(height[left],  
height[right]);
```

```
        maxArea = Math.max(maxArea, width *  
h);  
  
        if(height[left] < height[right]) {  
            left++;  
        } else {  
            right--;  
        }  
    }  
  
    return maxArea;  
}
```

### Dry Run

```
height = [1,8,6,2,5,4,8,3,7]
```

```
left=0  
right=8
```

```
Area = 8 × min(1,7)  
      = 8
```

```
Move left
```

```
left=1  
right=8
```

```
Area = 7 × min(8,7)  
      = 49
```

```
Maximum Area = 49
```

# TWO POINTERS –

## PROBLEM 4: MOVE ZEROES

### Problem Description

Move all zeros to the end of the array while maintaining the relative order of non-zero elements.  
Do it in-place.

### Sample Input

```
nums = [0,1,0,3,12]
```

### Sample Output

```
[1,3,12,0,0]
```

### Constraints

```
1 <= nums.length <= 10^4
```

### Brute Force Approach

#### Idea

Store all non-zero values in a temporary array and then fill remaining positions with zeros.

#### Time Complexity

$O(n)$

#### Space Complexity

$O(n)$

### Optimized Two Pointer Approach

#### Idea

Use:

- left → next position for non-zero element
- right → traversal pointer

Whenever non-zero found, swap.

#### Time Complexity

$O(n)$

#### Space Complexity

$O(1)$

### Company Tags

Facebook, Bloomberg, Amazon

### Practice Link

<https://leetcode.com/problems/move-zeroes/>

### Video Solution

<https://www.youtube.com/watch?v=aayNRwUN3Do>

### Java Solution

```
class Solution {
    public void moveZeroes(int[] nums) {

        int left = 0;

        for(int right = 0; right < nums.length;
            right++) {

            if(nums[right] != 0) {

                int temp = nums[left];
                nums[left] = nums[right];
                nums[right] = temp;

                left++;
            }
        }
    }
}
```

```
}
```

### Dry Run

```
nums = [0,1,0,3,12]
```

```
Swap 1 with 0
[1,0,0,3,12]
```

```
Swap 3
[1,3,0,0,12]
```

```
Swap 12
[1,3,12,0,0]
```

# TWO POINTERS –

## PROBLEM 5: REMOVE DUPLICATES FROM SORTED ARRAY

### Problem Description

Given a sorted array, remove duplicates in-place such that each unique element appears only once.

Return the number of unique elements.

### Sample Input

```
nums = [1,1,2]
```

### Sample Output

```
2
```

```
nums = [1,2,_]
```

### Constraints

```
1 <= nums.length <= 3 * 10^4  
-100 <= nums[i] <= 100
```

### Brute Force Approach

#### Idea

Use a HashSet to store unique elements.

Copy them back to the array.

#### Time Complexity

$O(n)$

#### Space Complexity

$O(n)$

### Optimized Two Pointer Approach

#### Idea

- left keeps track of last unique element.
- right scans the array.
- When a new element is found, place it after left.

#### Time Complexity

$O(n)$

#### Space Complexity

$O(1)$

### Company Tags

Amazon, Apple, Microsoft

### Practice Link

<https://leetcode.com/problems/remove-duplicates-from-sorted-array/>

### Video Solution

<https://www.youtube.com/watch?v=DEJAZBq0FDA>

### Java Solution

```
class Solution {  
    public int removeDuplicates(int[] nums) {  
  
        if(nums.length == 0) {  
            return 0;  
        }  
  
        int left = 0;  
  
        for(int right = 1; right < nums.length; right++) {  
  
            if(nums[right] != nums[left]) {  
                left++;  
                nums[left] = nums[right];  
            }  
        }  
  
        return left + 1;  
    }  
}
```

### Dry Run

```
nums = [1,1,2]
```

```
left = 0
```

```
right = 1
```

```
1 == 1
```

```
Skip
```

```
right = 2
```

```
2 != 1
```

```
left++
```

```
nums[1] = 2
```

```
Result:
```

```
[1,2,_]
```

```
Answer = 2
```

### Pattern Summary

| Problem                        | Two Pointer Type            |
|--------------------------------|-----------------------------|
| Two Sum II                     | Opposite Direction          |
| Valid Palindrome               | Opposite Direction          |
| Container With Most Water      | Opposite Direction + Greedy |
| Move Zeroes                    | Slow & Fast Pointer         |
| Remove Duplicates Sorted Array | Read & Write Pointer        |